Тема 1. Теоретическое задание (выполнил Павел Григоренко)

1. В бестиповом λ-исчислении [1, §5] выполните задания по подстановке с избежанием захвата, строгому вычислению и безымянному представлению.

   (a) Выполните подстановку `[y ↦ (λa. x a)(λb. z b)](λx. λw. λz. y x z)`.
   При необходимости выполните α-конверсию (переименование связанных переменных) и укажите итоговый результат подстановки.

   (b) Переведите следующий терм в безымянное представление (индексы де Брёйна) [1, §6] и вычислите его в безымянной форме, используя *строгое* вычисление (один шаг — один переход ($\longrightarrow$) по операционной семантике).

   ```
   (λf. λx. f (f (f x))) (λg. λy. g (g y)) (λz. z)
   ```

   Решение.

   (a)
   ```
   [y ↦ (λa. x a)(λb. z b)](λx. λw. λz. y x z)
   [y ↦ (λa. x a)(λb. z b)](λx. λw. λt. y x t)
   [y ↦ (λa. x a)(λb. z b)](λu. λw. λt. y u t)
   λu. λw. λt. (λa. x a)(λb. z b) u t
   ```

   (b)
   ```
   (λf. λx. f (f (f x))) (λg. λy. g (g y)) (λz. z)
   (λ λ 1 (1 (1 0))) (λ λ 1 (1 0)) (λ 0)
   (λ (λ λ 1 (1 0)) ((λ λ 1 (1 0)) ((λ λ 1 (1 0)) 0))) (λ 0)
   (λ λ 1 (1 0)) ((λ λ 1 (1 0)) ((λ λ 1 (1 0)) (λ 0)))
   (λ λ 1 (1 0)) ((λ λ 1 (1 0)) (λ (λ 0) ((λ 0) 0)))
   (λ λ 1 (1 0)) ((λ λ 1 (1 0)) (λ (λ 0) 0))
   (λ λ 1 (1 0)) ((λ λ 1 (1 0)) (λ 0))
   (λ λ 1 (1 0)) (λ (λ 0) ((λ 0) 0))
   (λ λ 1 (1 0)) (λ (λ 0) 0)
   (λ λ 1 (1 0)) (λ 0)
   λ (λ 0) ((λ 0) 0)
   λ (λ 0) 0
   λ 0
   ```

2. Для бестипового λ-исчисления с булевыми и арифметическими выражениями [1, §5] докажите по *индукции по правилам вывода* одношагового вычисления, что при вычислении число свободных переменных не увеличивается:

$$\text{если } t \longrightarrow t', \text{ то } FV(t') \subseteq FV(t) \quad (\text{в частности, } |FV(t')| \leq |FV(t)|).$$

Решение.

- Набор свободных переменных в if это объединение наборов свободных переменных его условия, и двух веток.
  - При выводе в одну из веток при явном значении условия, остаёмся лишь с набором свободных переменных этой ветки. Очевидно, что подмножество.
  - При выводе, когда условие выводится в новую форму, (по индуктивной гипотезе) множество его свободных переменных становится нестрогим подмножеством того, что было раньше. Очевидно, что если его теперь объединит с множествами для веток, получим подмножество того, что было раннее.
- `pred 0` $\mapsto$ `0`: было и осталось $\emptyset$.
- `pred (succ v)` $\mapsto$ `v`: было и осталось $FV(v)$.
- `iszero 0` $\mapsto$ `true`: было и осталось $\emptyset$.
- `iszero (succ v)` $\mapsto$ `false`: $\emptyset \subseteq FV(v)$.
- Правила вывода `pred`, `succ` и `iszero` при выводе их аргумента очевидно подходят по индуктивной гипотезе.
- Правила вывода аппликации при выводе её левой или её правой части тоже очевидно подходят по индуктивной гипотезе.
- Вычисление лямбды подстановкой аргумента. Будем считать, что ни одна абстракция в левой части не использует имена каких-либо свободных переменных правой части. Иначе выполним α преобразование. Теперь все новые вхождения правой части в итоговое выражение очевидно не поменяли своё множество свободных переменных. Что касается левой части, в её теле мы убрали все вхождения той переменной, по которой шла абстракция и саму абстракцию по этой переменной. Так что по определению свободных переменных в абстракции тоже очевидно нет изменений.

3. Приведите терм `t` в бестиповом λ-исчислении с арифметическими выражениями [1, §5], такой что

   - размер терма $\mathsf{size}(\texttt{t}) \geq 20$
   - `t (λx. λy. x)` $\longrightarrow^*$ `succ 0`
   - `t (λx. λy. y)` $\longrightarrow^*$ `succ (succ 0)`

   Решение.

```
t := λf. (pred (succ (pred (succ (pred (succ (pred (succ (pred (succ (pred (succ
(pred (succ (pred (succ (pred (succ (pred (succ (f (succ 0) (succ (succ 0))))))))))))))))))))
```

4. Преобразуйте фрагмент программы на Python ниже в эквивалентный фрагмент, применив операцию инлайнинга функций `make_reporter` и `report_cost`. Если для корректного преобразования требуется переименование переменных или другие изменения — явно дайте пояснение этим действиям. Убедитесь в корректности преобразования проверкой в интерпретаторе.

```
def make_reporter(level):
  threshold = level * 2
  def report_cost(msg_id):
    if msg_id >= 2:
      return (level * msg_id) + threshold
    return level * msg_id
  return report_cost

def total_diagnostic_cost():
  levels = list(map(int, input().split()))
  report_order = list(map(int, input().split()))
  msg_id = int(input())

  reporters = {}
  for level in levels:
    reporters[level] = make_reporter(level)

  total = 0
  for lev in report_order:
    total += reporters[lev](msg_id)

  return total

total_diagnostic_cost()
```

Решение.

Переименовываем аргумент у `report_cost` из `msg_id` в `m`. Меняем её две ветки на тернарник, и превращаем саму функцию в лямбду. Подставляем `report_cost` на место её вызова. Меняем возврат значения на запись его туда, куда раньше оно записывалось после возврата.

```
def total_diagnostic_cost():
  levels = list(map(int, input().split()))
  report_order = list(map(int, input().split()))
  msg_id = int(input())

  reporters = {}
  for level in levels:
    threshold = level * 2
    reporters[level] = lambda m: (level * m) + threshold if m >= 2 else level * m

  total = 0
  for lev in report_order:
    total += reporters[lev](msg_id)

  return total

total_diagnostic_cost()
```

Список литературы

[1] Б. Пирс. *Типы в языках программирования: пер. с англ.* Лямбда пресс, 2012. ISBN: 9785791300829. URL: https://books.google.ru/books?id=HJJCkgEACAAJ.